M2M challenges



Course title: Smart Industries
Date: 23-09-2026
Student: Gijs Brugmans

Table of contents

1. Revision history 3
2. Introduction 4
3. Exercises 5
   3.1 Exercise 1 5
   3.2 Exercise 2 8
   3.3 Exercise 3/4 10
   3.4 Exercise 6/7 12
   3.5 Exercise 8 14
   3.6 Exercise 9 15
   3.7 Exercise 10 17
   3.8 Exercise 12 18
   3.9 Exercise 13 19
   3.10 Exercise 14 20
   3.11 Exercise 16 21

1. Revision history

Date	Revision
15-09-2026	Included the first couple of exercises inside the report
16-09-2026	Included all the exercises until 9 into the report
22-09-2026	Added exercises 9 and 10 into the report
23-09-2026	Added exercises 12, 13, 14 and 16 into the report

2. Introduction

3. Exercises

In this part of the report the exercises of M2M will be shown in the different steps taken in the exercises will be discussed

3.1 Exercise 1

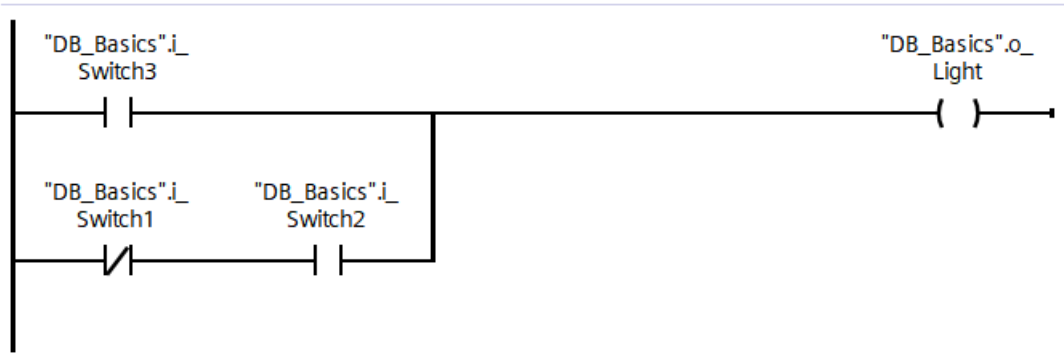
In this exercise a process has to be simulated where 3 switches interact with each other, and a light has to activate or deactivate when certain switches are activated or deactivated. These combinations are shown below

Combination	i_Switch1	i_Switch2	i_Switch3	o_Light
Combination1	TRUE	TRUE	TRUE	TRUE
Combination2	FALSE	TRUE	TRUE	TRUE
Combination3	TRUE	FALSE	TRUE	TRUE
Combination4	FALSE	FALSE	TRUE	TRUE
Combination5	TRUE	TRUE	FALSE	FALSE
Combination6	FALSE	TRUE	FALSE	TRUE
Combination7	TRUE	FALSE	FALSE	FALSE
Combination8	FALSE	FALSE	FALSE	FALSE

Table 4.1 – Truth-table function Q_light

Picture 1: In the picture above the truth-table of this simulation is shown

Now in the pictures below first the system will be shown and after that proof that every combination does what it has to do.

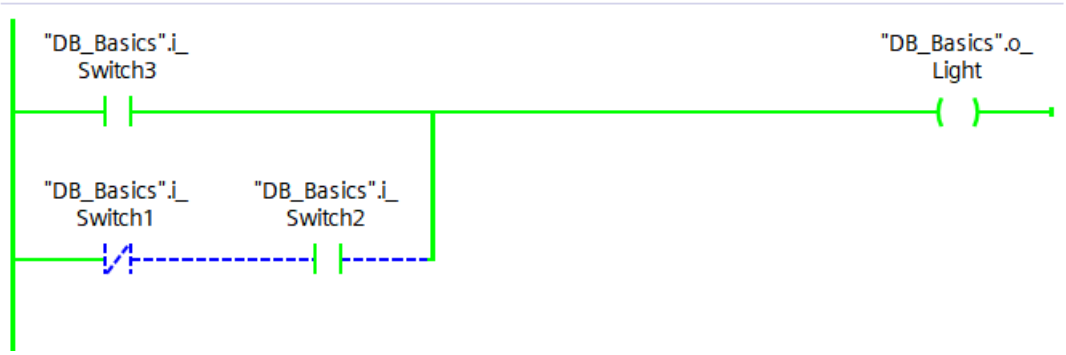


Picture 2: In the picture above the final design of the simulated system is shown

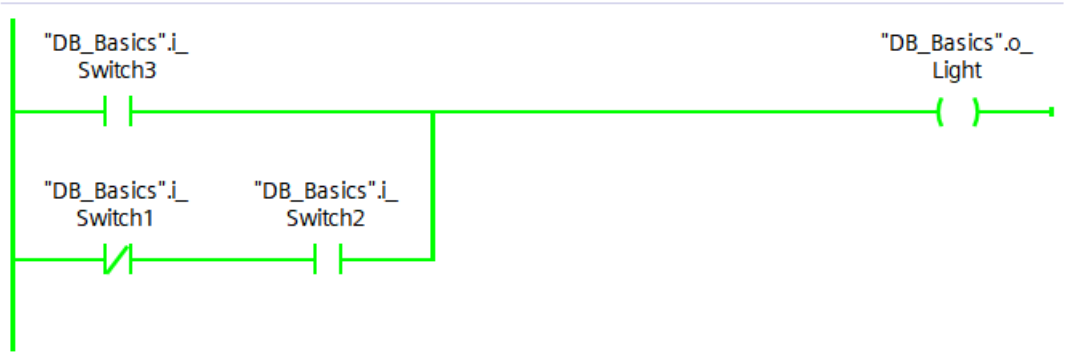
In the picture above the whole simulation is shown. In this simulation it is visible that switch 1 and switch 2 are connected to each other in an AND sequence and switch 3 is connected to AND sequence of switch 1 and switch 2 with an OR sequence.

This is because every time switch 3 is activated the light also needs to be activated. Therefore, it stands alone, but there is also a sequence where switch 3 is deactivated but the light still activates. In this sequence switch 2 is activated and switch 1 is deactivated. Because of this the output of switch 1 is inverted to let power run through when it is deactivated and when switch 2 is then activated it lets the light be activated

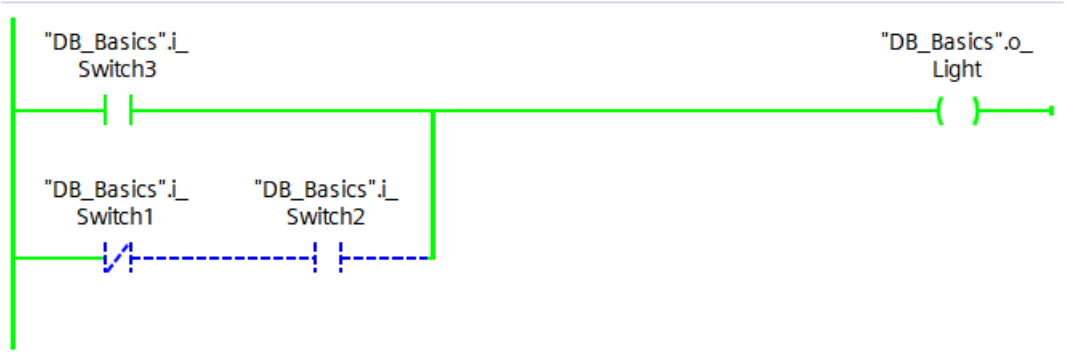
In the next page the proof that the different combinations all work correctly will be shown.



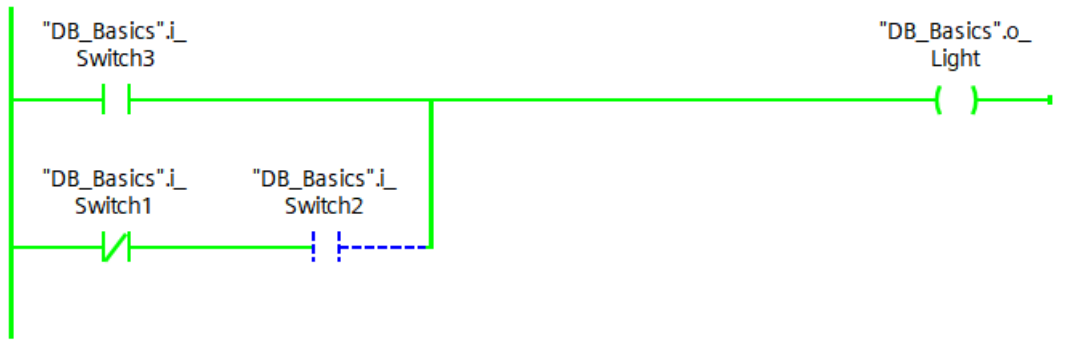
Picture 3: In the picture above the solution for the first combination is shown where the light needs to be on



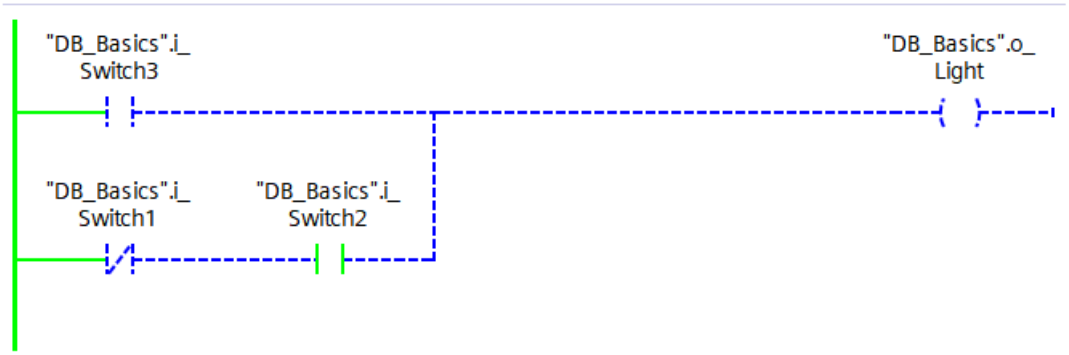
Picture 4: In the picture above the solution for the second combination is shown where the light needs to be on



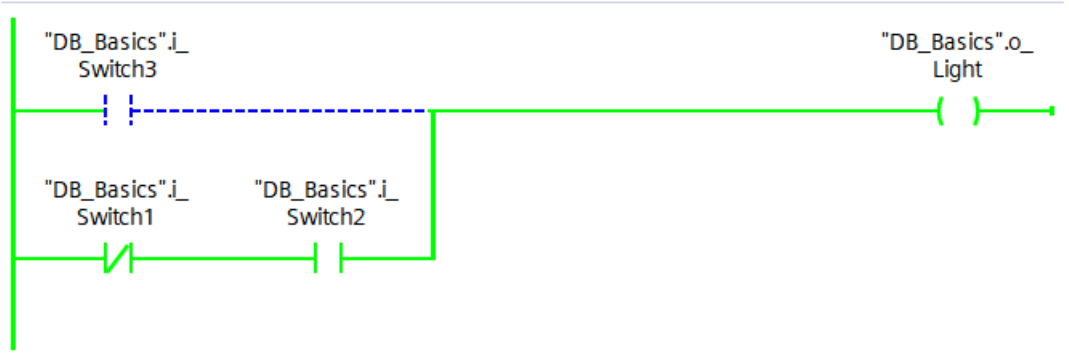
Picture 5: In the picture above the solution for the third combination is shown where the light needs to be on



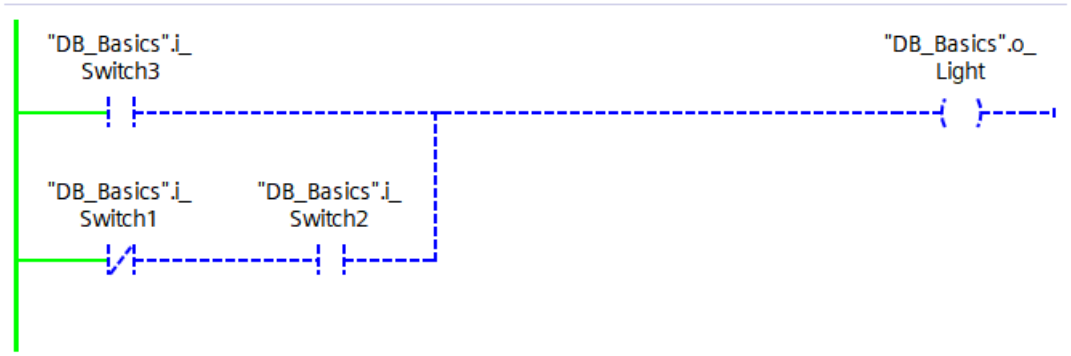
Picture 6: In the picture above the solution for the fourth combination is shown where the light needs to be on



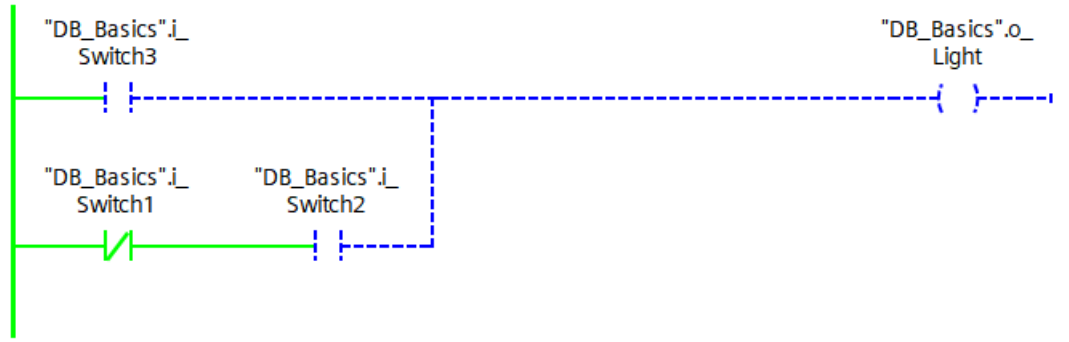
Picture 7: In the picture above the solution for the fifth combination is shown where the light needs to be off



Picture 8: In the picture above the solution for the sixth combination is shown where the light needs to be on



Picture 9: In the picture above the solution for the seventh combination is shown where the light needs to be off



Picture 10: In the picture above the solution for the eighth combination is shown where the light needs to be off

3.2 Exercise 2

In this exercise four different simulations had to be made, that are shown below

$$o_Out1 = i_In1 * (\overline{i_In2 + i_In3})$$

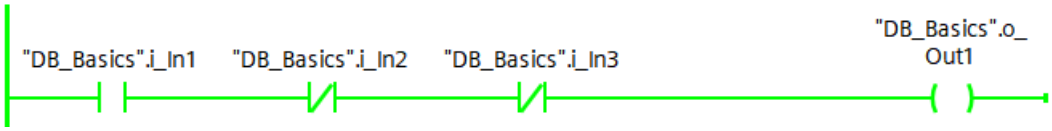
$$o_Out2 = i_In4 + (\overline{i_In3 * i_In1})$$

$$o_Out3 = i_In1 + (i_In2 * i_In3)$$

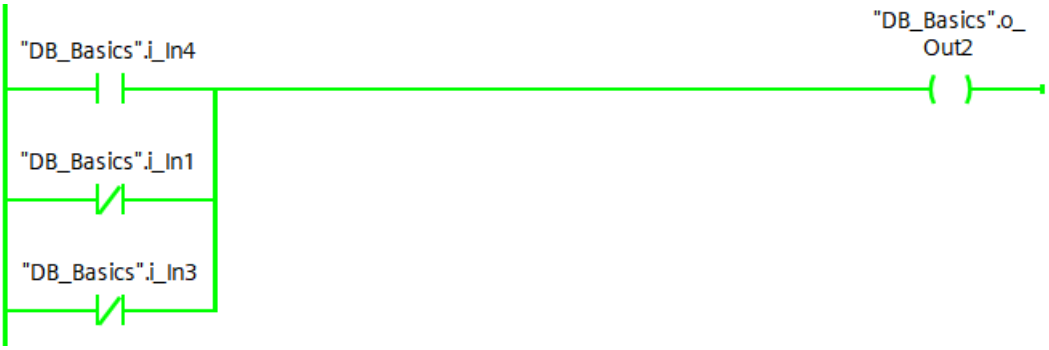
$$o_Out4 = i_In4 * (\overline{i_In2 + i_In3})$$

Picture 11: In the picture above the four different simulation designs are shown that have to be made during this exercise

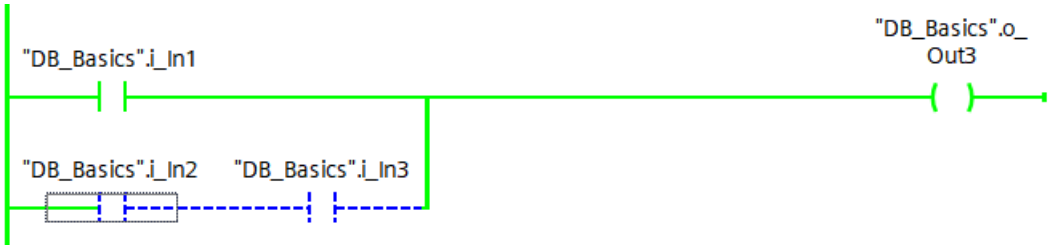
These outputs will be shown in the pictures below



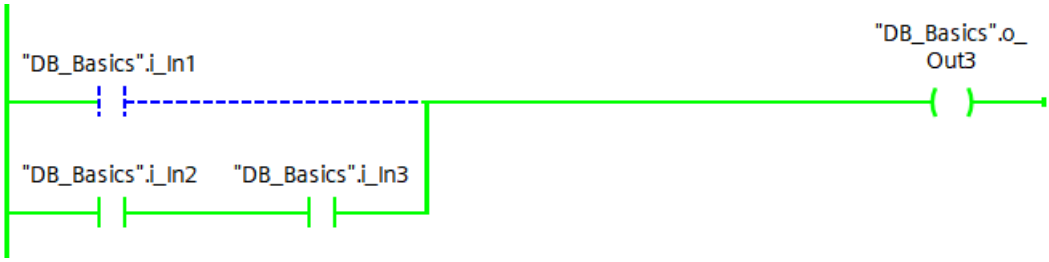
Picture 12: In the picture above the solution of the first design is shown



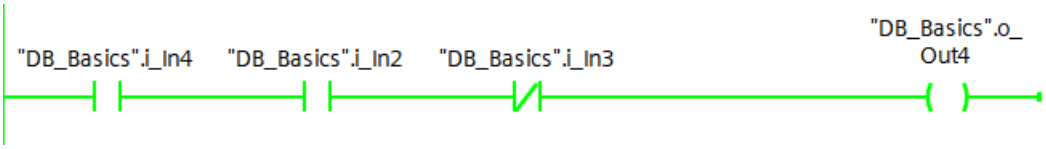
Picture 13: In the picture above the solution of the second design is shown



Picture 14: In the picture above the solution of the third design is shown where only input 1 is activated and input 2 and input 3 are deactivated



Picture 15: In the picture above the solution of the first design is shown where input 2 and input 3 are activated and input 1 is deactivated.



Picture 16: In the picture above the solution of the fourth design is shown

3.3 Exercise 3/4

In exercise 3 a system had to be simulated that activates a motor and can only deactivate the motor if the motor is already activated. For this system the following design was given that is shown in the picture below:

$$o_Motor1 = i_Start1 + (o_Motor1 \cdot \overline{i_Stop})$$

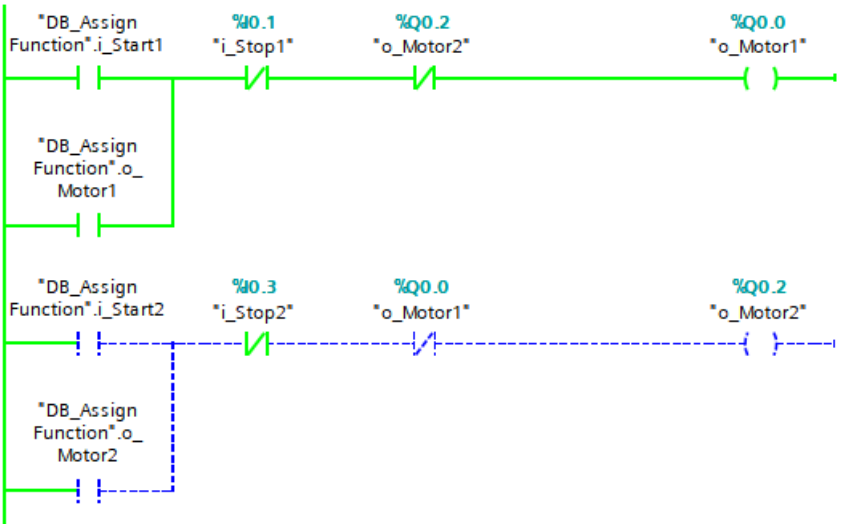
Picture 17: In the picture above the design of exercise 3 is shown.

In exercise 4 this gets improved upon by including a second motor. In this system only one of the motors can run at any given time, so if motor 1 is activated then motor cannot be activated and the reverse is also true. This design is shown in the picture below:

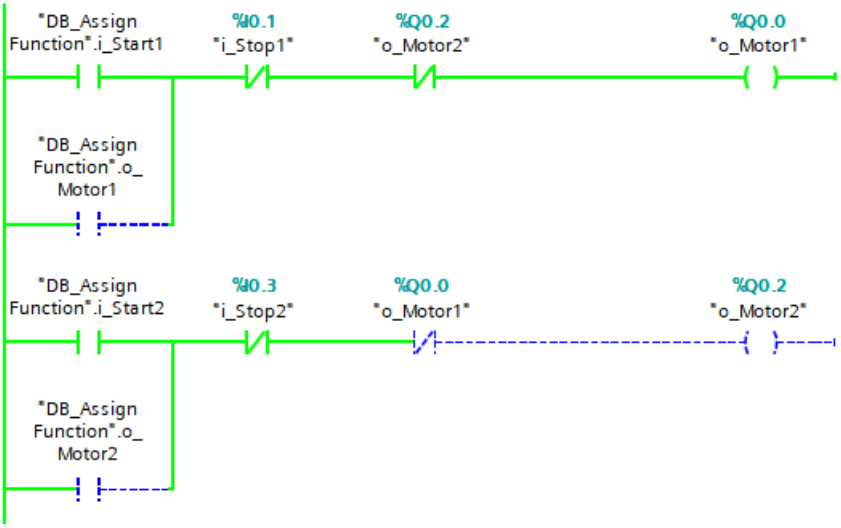
- (i_Start1 **OR** (o_Motor1 **AND NOT** i_Stop1)) **AND NOT** o_Motor2 = o_Motor1 (active)
- (i_Start2 **OR** (o_Motor2 **AND NOT** i_Stop2)) **AND NOT** o_Motor1 = o_Motor2 (active)

Picture 18: In the picture above the design of exercise 4 is shown

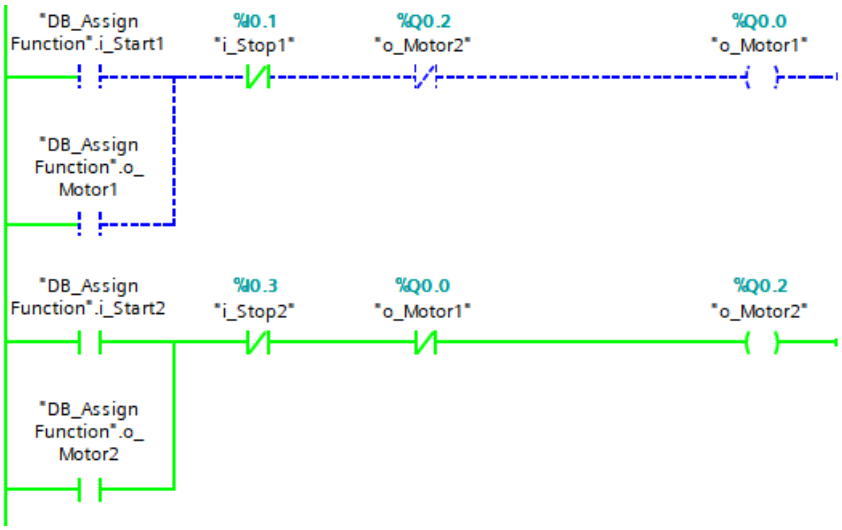
The solution for this whole exercise is shown below



Picture 19: In the picture above the solution of exercise 4 is shown where only motor 1 is activated and therefore running while motor 2 is deactivated and not running



Picture 20: In the picture above the solution of exercise 4 is shown where only motor 1 is activated and therefore running while motor 2 is trying to be activated, but because motor 1 is already activated it is not running



Picture 21: In the picture above the solution of exercise 4 is shown where only motor 2 is activated and therefore running while motor 1 is deactivated and not running

3.4 Exercise 6/7

In this exercise a new function had to be made that contained two different behaviors inside of this function and these behaviors had to be structured like their design shown in the picture below:

$$o_InterMotor1 = i_InterStartButton1 * (\overline{i_InterStartButton2} * \overline{i_InterStopButton})$$

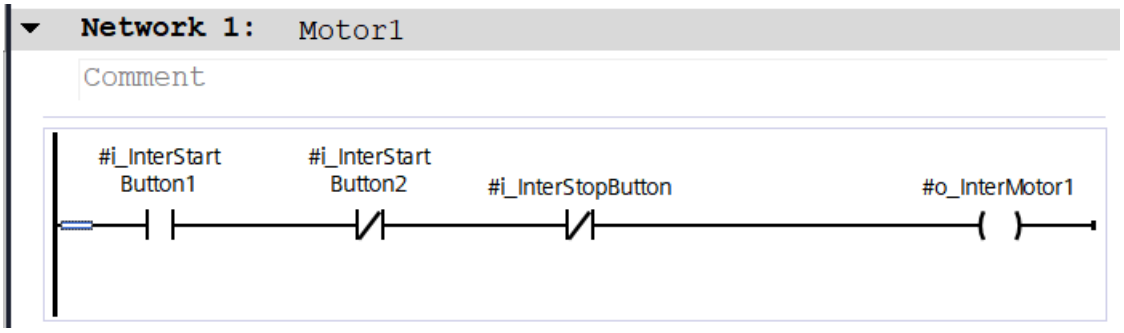
$$o_InterMotor2 = i_InterStartButton2 * (\overline{i_InterStartButton1} * \overline{i_InterStopButton})$$

Picture 22: In the picture above the design of the networks that have to be inside the own made function are shown

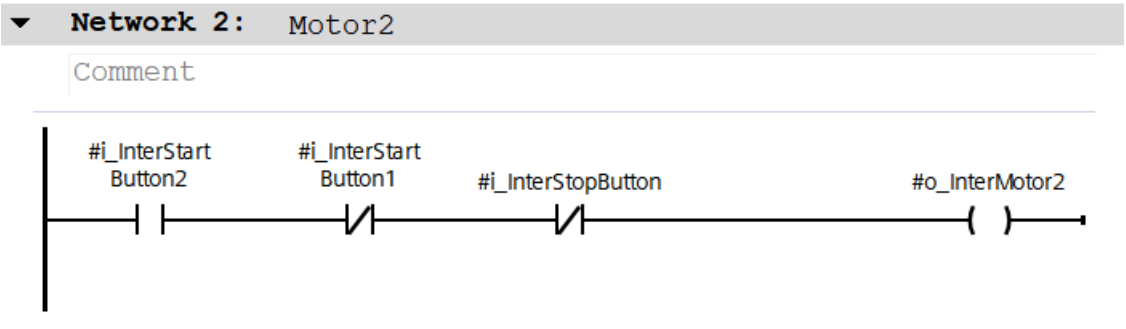
Now the implemented function will be shown

		Name	Data type	Default value	Comment
1		Input			
2		i_InterStartButton1	Bool		
3		i_InterStartButton2	Bool		
4		i_InterStopButton	Bool		
5		Output			
6		o_InterMotor1	Bool		
7		o_InterMotor2	Bool		

Picture 23: In the picture above the different inputs and outputs of the new function are shown

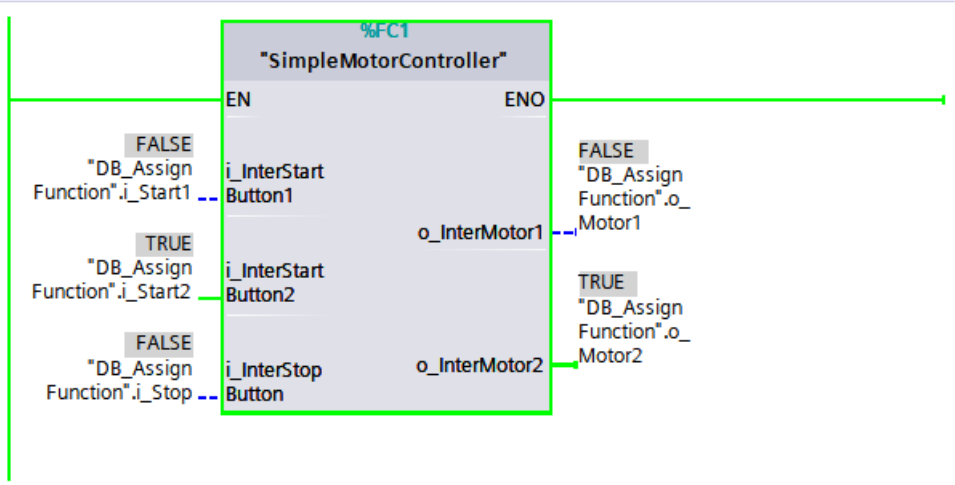


Picture 24: In the picture above the network of motor 1 is shown



Picture 25: In the picture above the network of motor 2 is shown

Finally a block interface had to be put down inside OB1 with the new value from this function as shown in picture 26 below to conclude exercise 6 and continue to exercise 7. In exercise 7 inputs and outputs are going to be assigned to the block interface. The product of this exercise is shown in picture 26 below

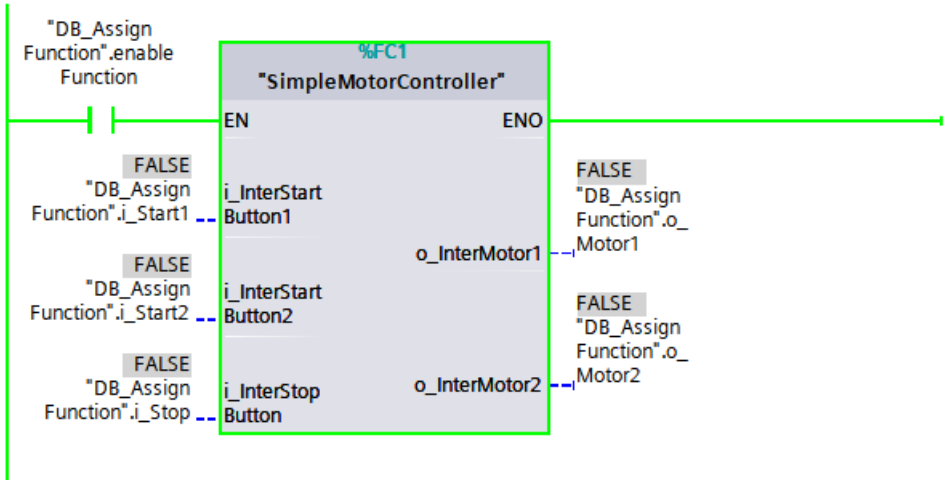


Picture 26: In the picture above the block interface is shown with all the inputs and outputs

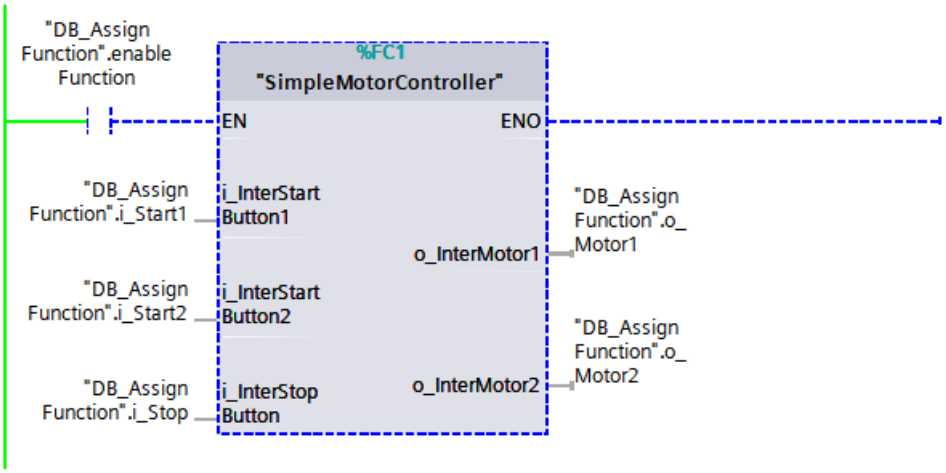
When changing the different inputs the different outputs also change. When button 1 gets changed from true to false while keeping button 2 also on for example both motors give a false output and when button 2 gets to give false as an input motor 1 starts to give a true output.

3.5 Exercise 8

In this exercise the same network has to be used as in exercise 6 and 7, but now an enableFunction value has to be made that enables the possibility to enable or disable the function to work. This is shown in the picture below.



Picture 27: In the picture above the enable function is implemented and activated, therefore activating the block structure and all of the assigned inputs and outputs



Picture 28: In the picture above the enable function is implemented but not activated, therefore the block structure with all the assigned inputs and outputs are also still deactivated

3.6 Exercise 9

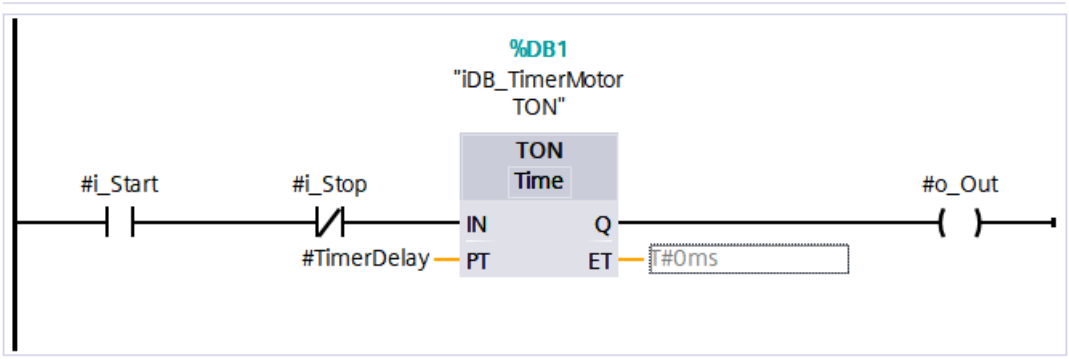
In this exercise a new project has to be made by yourself. First a function block for a timerdelay has to be made which is implemented in the pictures below

TimedMotorController

		Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...
1		Input						
2		i_Start	Bool	false	Non-retain	☑	☑	☑
3		i_Stop	Bool	false	Non-retain	☑	☑	☑
4		TimerDelay	Time	T#0ms	Non-retain	☑	☑	☑
5		<Add new>						
6		Output						
7		o_Out	Bool	false	Non-retain	☑	☑	☑

Picture 29: In the picture above the inputs and outputs of the function block are shown

With the implementation it was specified that the output can only be given if Start is activated and Stop is deactivated, which gives the implementation that is shown in picture 30



Picture 30: In the picture above the implementation of the FB timerdelay is shown

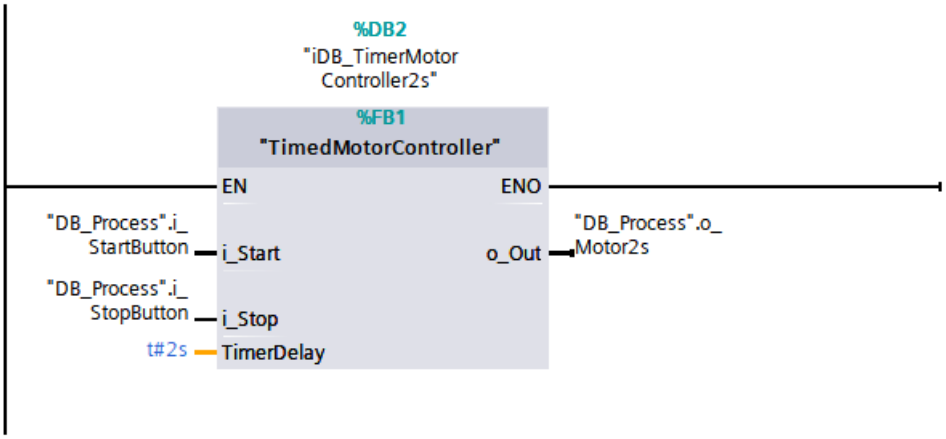
After this a database had to be made in order to regulate and control the timer. This database contains the following values as showed below.

DB_Process

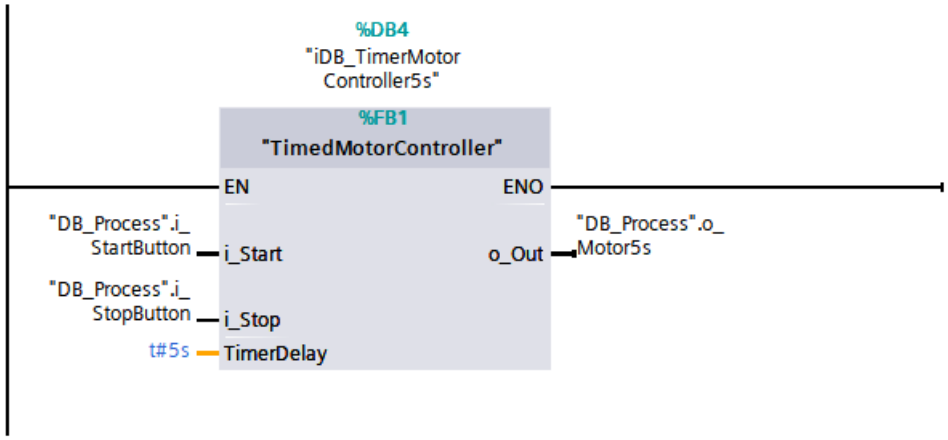
		Name	Data type	Start value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint
1		Static							
2		i_StartButton	Bool	false		☑	☑	☑	
3		i_StopButton	Bool	false		☑	☑	☑	
4		o_Motor2s	Bool	false		☑	☑	☑	
5		o_Motor5s	Bool	false		☑	☑	☑	

Picture 31: In the picture above the implementation of the database is shown

After this the actual timers could be implemented as shown in picture 32 and picture 33. One thing that was undeniable was that the timer wasn't working correctly when trying it out.



Picture 32: In the picture above the implemented timer is shown where it should have a delay of 2 seconds

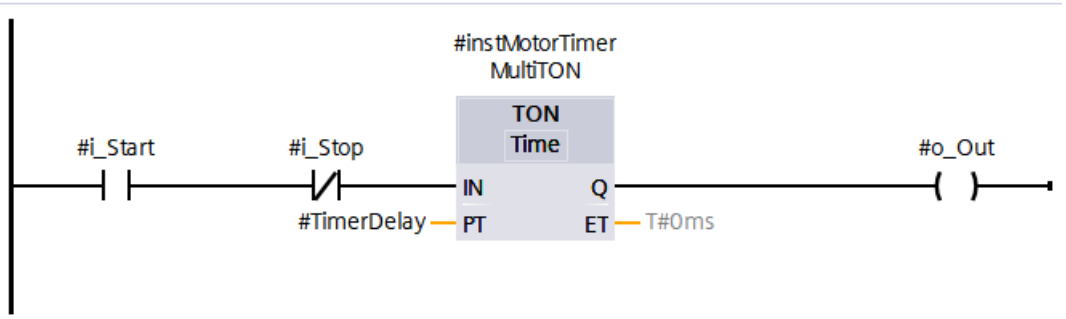


Picture 33: In the picture above the implemented timer is shown where it should have a delay of 5 seconds

3.7 Exercise 10

In this exercise the timer will be made so that it actually works as a timer and gives the correct delay.

This is done by making the TON timer that was made during the previous exercise into a multi instance timer instead of a single instance



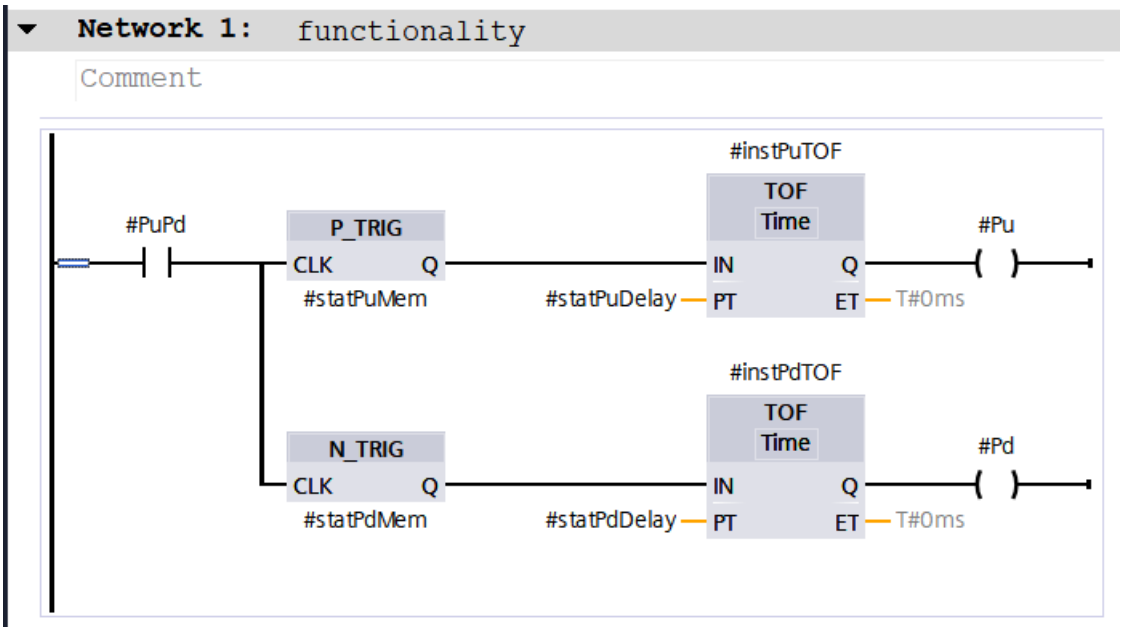
Picture 34: In the picture above the new changed TON timer implementation is shown.

Because of the changed implementation the networks that were shown in pictures 32 and 33 now actually activate with the correct delay of 2 seconds for the network in picture 32 and 5 second in the network in picture 33

3.8 Exercise 12

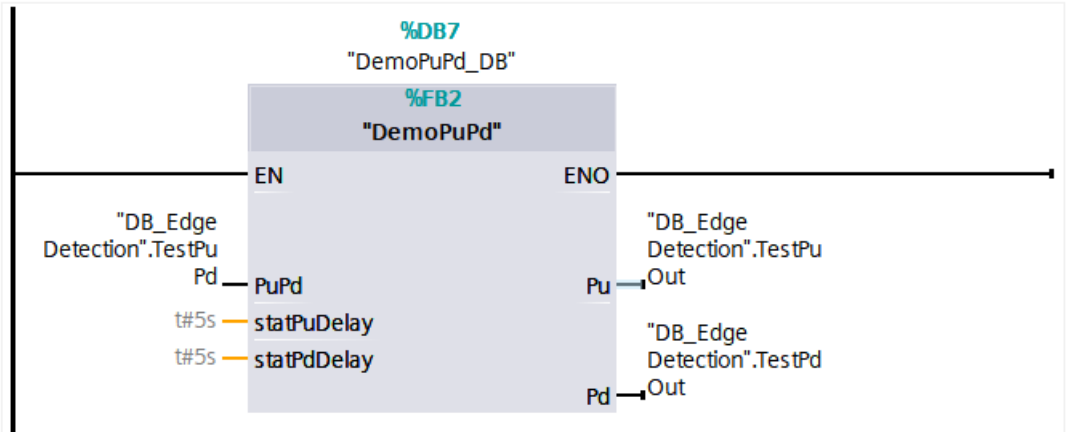
In this exercise a technique is being mentioned to detect ascending and descending flanks in a system. It is going to be investigated how the rising_edge and falling_edge detection are implemented in a program and how they behave.

For this a new FB had to made that includes two timers where either a Pu (Pulse up) or Pd (Pulse down) is activated for a determined amount of time, which in the case of the exercise will be 5 seconds. The implementation of this FB is shown in picture 35 below.



Picture 35: In the picture above the FB is shown where two timers activate Pu and Pd for a certain amount of time before deactivating them again

This FB then got called in OB1 and connected to a database that gives the function block signals that actually work. This is shown in picture 36 below



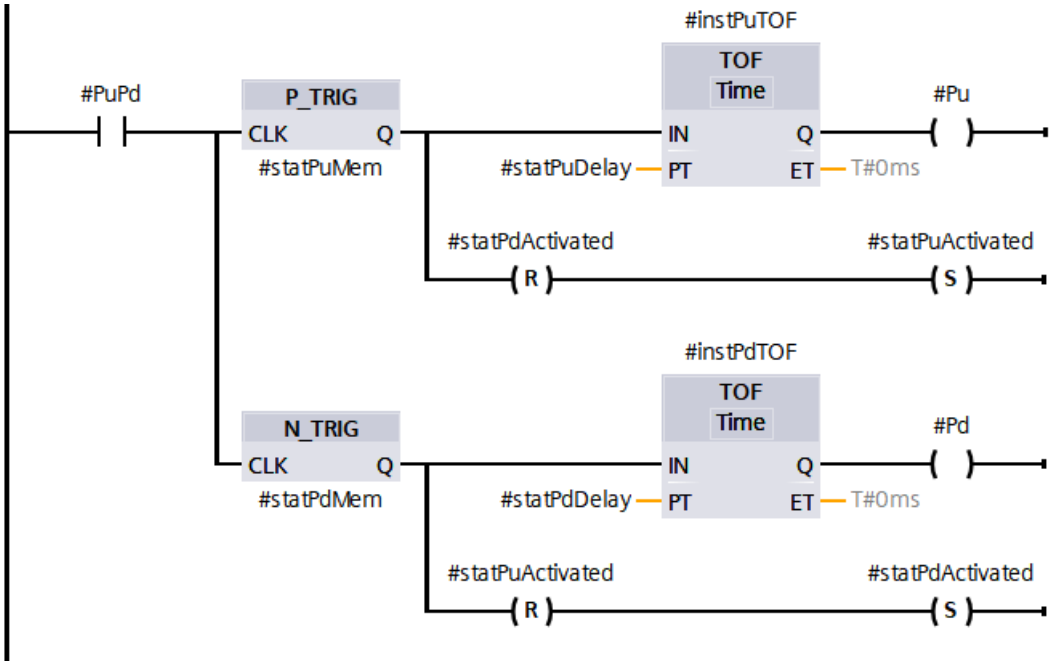
Picture 36: In the picture above the implementation of the FB is shown with the correct signals that it needs to work

3.9 Exercise 13

In this exercise the set and reset will be used for the first time. This exercise will use the program that was made during exercise 12 as a base to further work with.

The set and reset will probably do what the name suggests in my opinion. The set will set the thing it calls to give a true as a result and the reset will reset the thing it calls back to give false as a result.

For this exercise two more variables have to be made and implemented into the FB that gives a structure like the one shown in picture 37 that is shown below.



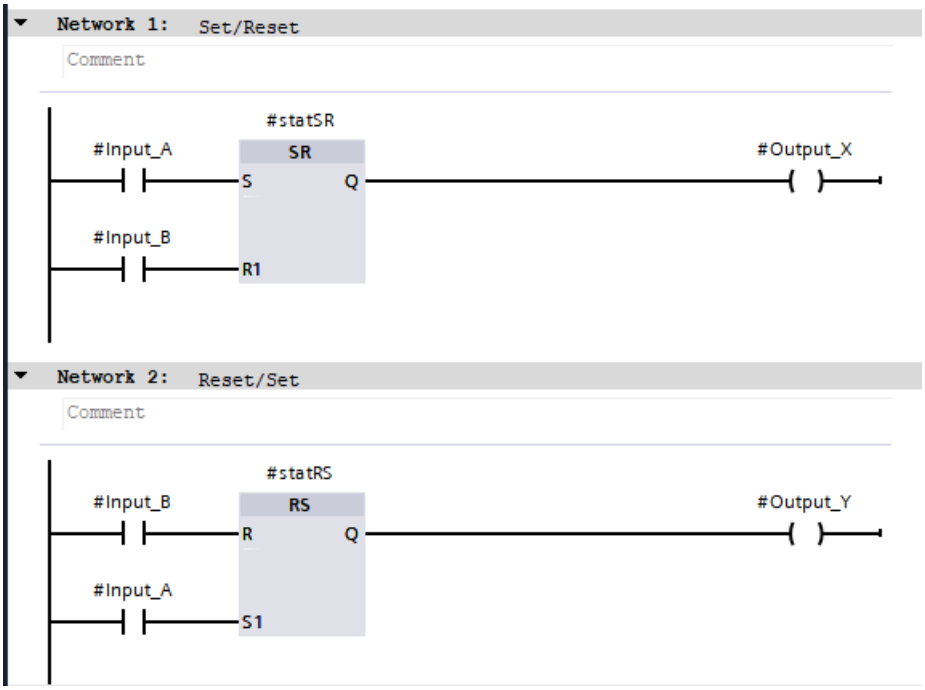
Picture 37: In the picture above the new structure using the set and reset functions and the new variables statPuActivated and statPdActivated

When running this program, it is visible that when the trigger switches the new states of Pu and Pd update immediately.

3.10 Exercise 14

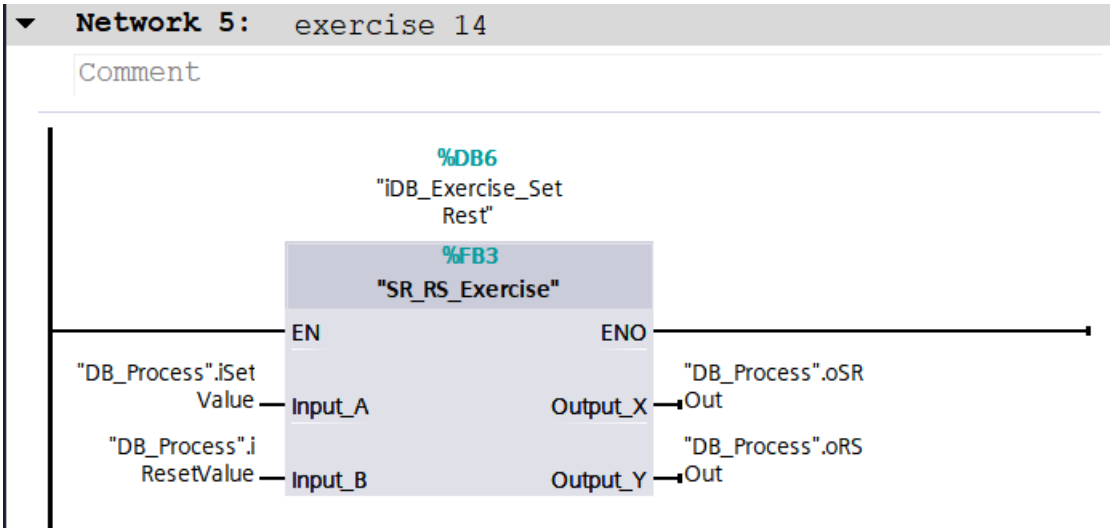
In this exercise an introduction was made to the difference of the SetReset and ResetSet functions.

Firstly a new FB with in there two networks had to be made. 1 network contained a SetReset function and the other network contained a ResetSet function. These were implemented like shown in picture 38 that stands below



Picture 38: In the picture above the SetReset and ResetSet functions are shown.

After this the FB had to be called inside OB1 where if you ran the program when implemented like how the program is shown in picture 39. When running the program I noticed that the difference between SetReset and ResetSet is the priority in which they occur. So in SetReset the Set will happen first because it has the higher priority of the Set and Reset and the inverse counts for ResetSet

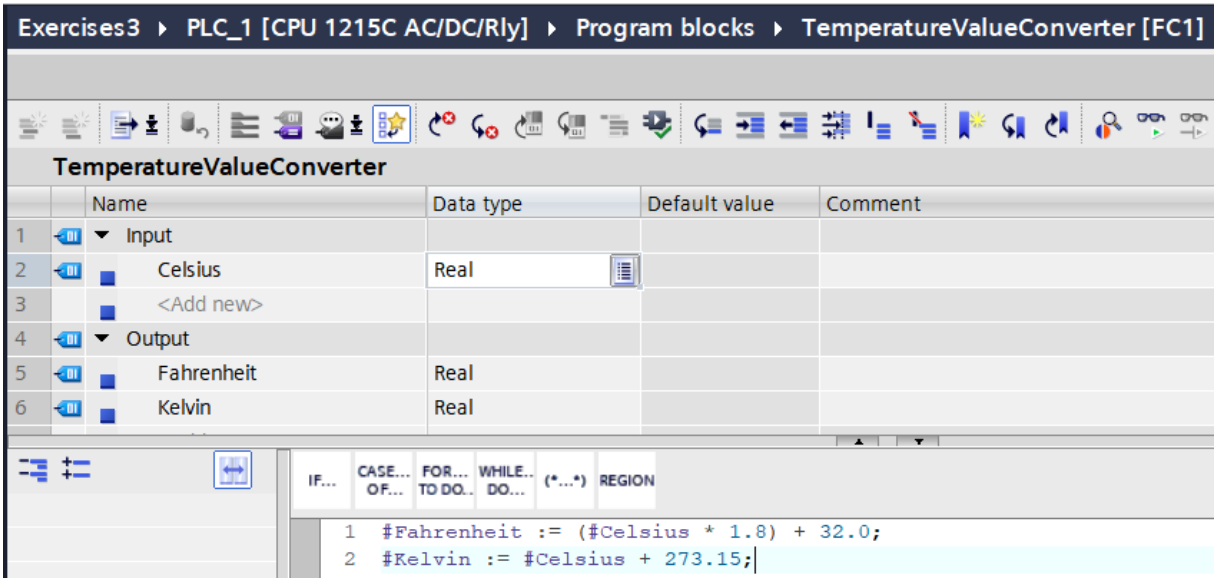


Picture 39: In the picture above it is shown how the FB gets called into the network inside OB1

3.11 Exercise 16

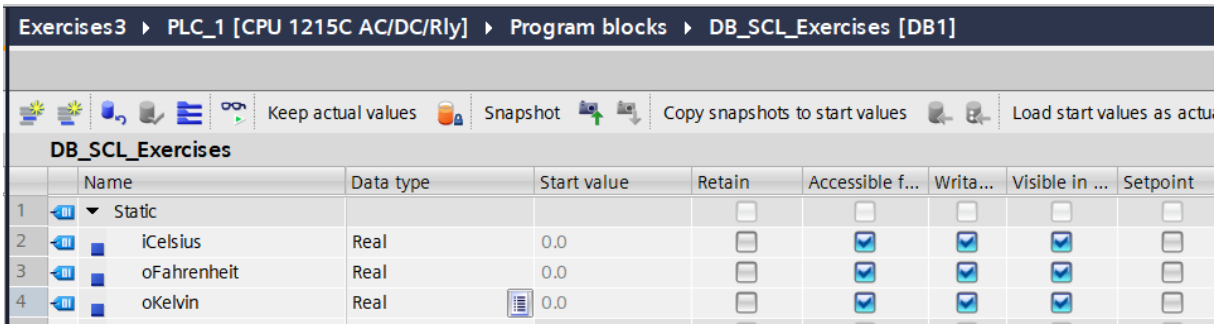
This is the first exercise where there is made use of SCL, which stands for Structured Control Language. In this exercise there are not a lot of clues given which made it a lot harder then I expected it to be, but the exercise that has to be done is to create a function that can convert the temperature in Celsius to Fahrenheit and Kelvin.

This was done to create a new function called TemperatureValueConverter where there is 1 input, Celsius, and two outputs, Fahrenheit and Kelvin. Then the formula's that were given in the exercise were made into some simple SCL which made it look like shown in picture 40 which is visible below



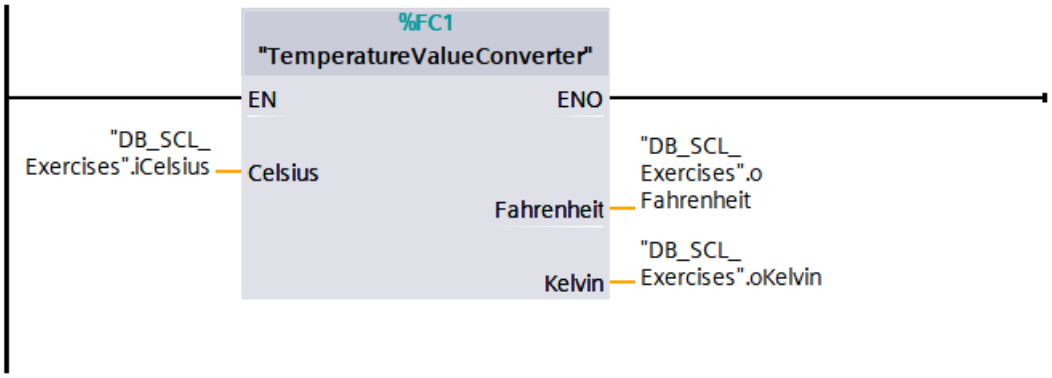
Picture 40: In the picture above the TemperatureValueConverter that is made with SCL is shown

To make sure that Celsius, Fahrenheit and Kelvin have actual values, a database was made to make sure that their values could be stored. This gave form to the database shown below in picture 41.

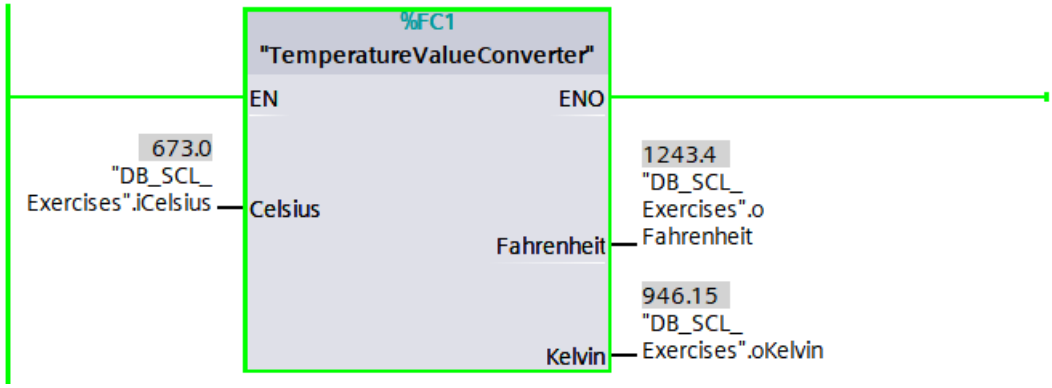


Picture 41: In the picture above the database DB_SCL_Exercises is shown with the input Celsius, output Fahrenheit and output Kelvin as variables

When the function got called into OB1 and the variables got connected it made for the implementation visible in picture 42 and the execution where the temperature values get converted shown in picture 43 both on the next page



Picture 42: In the picture the implementation of the FC getting called in OB1 together with the variables from the database is shown



Picture 43: In the picture above the execution of the temperature value converter is shown